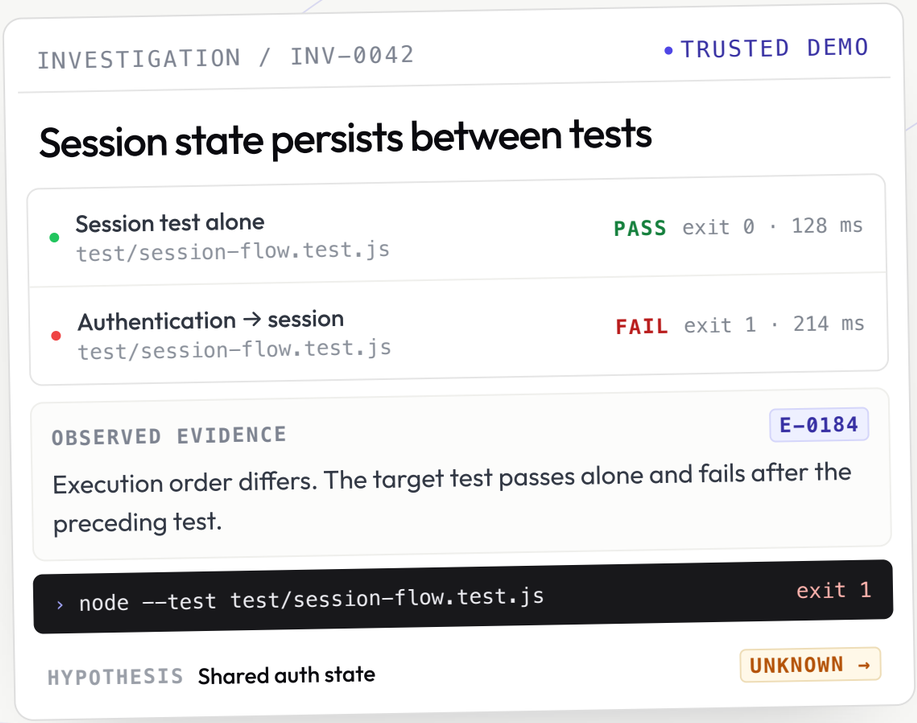
• EVIDENCE-FIRST DEBUGGING

Don't guess the bug. *Prove* it.

A local debugging workspace and evidence engine for failures that only happen *in context*. Run the test. Compare the runs. Keep every claim attached to its source.

• REAL TEST EXECUTION • RUN COMPARISON • EVIDENCE-LINKED HYPOTHESES
• VERIFIED FIXES



EXECUTION → EVIDENCE → VERDICT

Tests pass alone. They fail in *context*.

A log tells you the test failed. It does not tell you what changed, which claim is supported, or whether a fix survived the same condition. Breakpoint records the condition, the difference and the decision.

CONTROLLED CONTEXT MATRIX

Same test. Five contexts. One variable.

sample-project / test/session-flow.test.js

01 / ISOLATED	02 / AUTH	03 / PAIR	04 / REVERSE	05 / REPEAT
• PASS	• PASS	• FAIL	• PASS	• FAIL
session alone	**auth alone**	**auth → session**	**session → auth**	**auth → session**
exit 0 · 128 ms	exit 0 · 91 ms	exit 1 · 214 ms	exit 0 · 136 ms	exit 1 · 208 ms
• session	• auth	• auth → session	• session → auth	• auth → session
no predecessor	no predecessor	preceded by auth	auth runs after	same pair, again
		ORDER		ORDER

ORDER

Auth runs before session only in the failing pair.

E-0184 · OBSERVED

OUTCOME

Target passes alone; fails after the preceding test.

E-0185 · OBSERVED

UNKNOWN

Application state was not captured. The mechanism is not proven.

U-0007 · EXPLICIT

THE SIGNAL

Only execution order tracks the failure.

Five controlled contexts. Same test, same source, one variable.

Observed, not inferred

NEXT → CAPTURE APPLICATION STATE

01 The condition

The exact command, order, environment and timestamp of every run.

RUN_0181 RUN_0184 RUN_0185

02 The difference

Passing and failing runs side by side, with findings linked to evidence IDs.

PASS FAIL E-0184 E-0185

03 The claim

Observed facts, a hypothesis with confidence, and an explicit unknown.

OBSERVED HYPOTHESIS UNKNOWN

One investigation. Four moves. *Real evidence.*

Breakpoint is not a chat about a bug. It is a workspace that runs the test, compares the runs, keeps evidence attached to claims, and verifies a patch with another execution.

THE CORE LOOP

Every claim should point back to a run.

The differentiator is not one feature. It is the connection between execution, comparison, diagnosis, patch review and verification.

01 / REPRODUCE — RUN RECORD

Run the condition.

Execute the approved test in a controlled context. Capture the command, output, exit code and timing.

- exit code · stdout · environment

02 / COMPARE — DIFF

Put runs side by side.

Find what actually differs between a passing and a failing run. Link each finding to evidence IDs.

- order · status · output · duration

03 / INVESTIGATE — CLAIM

Separate fact from story.

State a possible cause with confidence, supporting evidence, alternatives and an explicit unknown.

- observed · hypothesis · unknown

04 / VERIFY — RESULT

Run it again.

Review a minimal patch, apply it to the approved source, and rerun the same condition. Record the actual result.

- verified · not verified · unknown

↺ Done = reproduce · compare · explain · patch · verify. Every step writes a durable record.

OBSERVED — • FACT

What the run data directly shows.

Execution order differs. The target test passes alone and fails after the preceding authentication test.

run_0181 · isolated	PASS
run_0184 · ordered	FAIL

SOURCE · E-0184 · E-0185

HYPOTHESIS — • LOW CONFIDENCE

What may explain those facts.

The failure is consistent with state established by the preceding test, but the captured runs do not prove the mechanism.

CONFIDENCE · LOW · 2 ALTERNATIVES

ALTERNATIVES · runner ordering · dependency behavior

UNKNOWN — • EXPLICIT

What has not been established.

Application state was not captured. Worker isolation and the exact state leak remain untested.

Application state	NOT CAPTURED
Worker isolation	UNTESTED

NEXT EXPERIMENT · add state capture, rerun

A small system with a *long memory*.

Next.js UI, route handlers, a controlled runner, an evidence graph and a durable record. AI is optional; it never outranks a run. Every artifact stays linked to the execution that produced it.

01 / INTERFACE — NEXT.JS

Investigation workspace

INV-0042 — Needs review

Session state persists between tests

Evidence | Run comparison | Fix review

5	3	1
RUNS	EVIDENCE	UNKNOWN

READ MODEL

The workspace renders recorded state. It cannot invent a run or send a command.

Reads snapshots — Writes actions, never commands

02 / ENGINE — POST /api/investigations

Controlled investigation

RUN Controlled runner
allowlist · spawn without shell · 10s · 64KB

DIFF Comparison engine
run pair · findings · certainty · evidence

AGENT Deterministic loop
6 tools · bounded · evidence-linked

PATCH Review gate
minimal diff · baseline · approved source

VERIFY Fresh run
passed → verified · failed → not verified

UI → action → allowlisted run → evidence → snapshot

Deterministic by default — AI is optional

03 / RECORD — .breakpoint

One inspectable history

.breakpoint/investigations.json — ATOMIC WRITE

Project	ROOT	Investigation	1:N
Test run	N	Evidence	N
Comparison	N	Hypothesis	N
Patch	N	Verification	1

Atomic replace prevents partial writes. Run artifacts stay embedded in the record.

Optional AI boundary
Server-only Responses API. Visible 503 when unavailable. No diagnosis is produced by the model.

Every claim points back to a run — Artifacts survive the chat

INVESTIGATION LIFECYCLE — persisted: open · needs_review · verified

OPEN → RUNNING → NEEDS REVIEW → VERIFIED ↳ NOT VERIFIED / UNKNOWN

A status is a fact about the record, not a claim about the cause.

RUN STATUS — execution truth

RUNNING → PASSED | FAILED | TIMEOUT | CANCELLED | INFRASTRUCTURE_ERROR

Only a completed execution can produce a terminal status.

The agent proposes. The run *testifies.*

A narrow tool vocabulary and a bounded loop. The agent proposes a hypothesis and a patch. The runner records what actually happened. Verification is another execution, not a stronger sentence.

01 / COMPARE — What the runs show

RUN PAIR

`run_0181` — • PASS

session alone

`exit 0 128 ms isolated`

`run_0184` — • FAIL

auth → session

`exit 1 214 ms ordered pair`

ORDER	Auth runs before session only in the failing pair.
OUTCOME	The target passes alone; it fails after the preceding test.
UNKNOWN	Application state was not captured. The mechanism is not proven.

SOURCE · E-0184 · E-0185 · ILLUSTRATIVE FIXTURE

02 / HYPOTHESIZE — What may explain it

LOW CONFIDENCE

`HYP-0007` — LOW

The session failure is consistent with state established by the preceding authentication test, but the captured runs do not prove the mechanism.

SUPPORTS	E-0184 · E-0185
ALTERNATIVES	Runner ordering · dependency behavior

NEXT EXPERIMENT

Add state capture or cleanup instrumentation, then rerun the same approved commands.

`A run that can support or reject the claim.`

CLAIM CONTRACT

Supported by evidence IDs. Limited by an explicit unknown. Falsifiable by the next run.

`Confidence rises only when new evidence arrives.`

03 / PATCH → VERIFY — What the next run says

VERIFIED

PROPOSED PATCH — • DRAFT

Reset module-level session state before every test.

```
+ import { clearSession } from "@/lib/session"
 ...
+ test.beforeEach(() => clearSession());
```

Review-only in the prototype. Application is gated to the approved demo source and baseline.

VERIFICATION RUN — • PASS

O **run_0186**

`same ordered condition · 201 ms`

✓ **VERIFIED** this patch passed this condition

This does not prove the mechanism. The next experiment is still state capture.

`VERIFICATION RULE · a pass proves this condition, not the root cause.`

BOUNDED LOOP

OBSERVE → CHOOSE TOOL → RECORD → STOP

TOOL VOCABULARY

`run_tests` `compare_runs` `inspect_test_output` `inspect_available_state` `record_hypothesis` `propose_experiment`

GUARDRAILS

No shell · no browser path · no free-form patch body. Optional AI is server-only.

Trust is the *feature*.

No fabricated logs. No invented causes. No unverified fixes. The final report is the product: what ran, what was observed, what remains unknown, and what changed.

EVALUATION

What must be true

- **Reproduce** — RUN
  Real command, exit code and captured output.
- **Compare** — EVIDENCE
  Findings linked to the runs that produced them.
- **Investigate** — CLAIM
  Observed, hypothesis and unknown kept separate.
- **Patch** — REVIEW
  Minimal diff, baseline check and review gate.
- **Verify** — PROOF
  A fresh run records the actual result.

NO ROOT CAUSE WITHOUT INSTRUMENTATION · NO PASS WITHOUT A RUN.

SECURITY & OBSERVABILITY

What cannot happen

01 **Allowlisted commands only**
No arbitrary test command or working directory from the browser.

02 **spawn without a shell**
No shell interpolation, no command string built from user input.

03 **10s timeout · 64KB cap**
Output is clipped and the process is killed on timeout.

04 **No free-form patch body**
Patches are reviewable diffs against the approved demo source.

05 **Optional AI, server-only**
Visible 503 when unavailable. No diagnosis is produced by the model.

TRUSTED LOCAL DEMO ONLY · not a sandbox · logs carry run ID, action, status and duration — never secrets.

BUILD ORDER & REPORT

What ships next

01 **Reproducible runner** — FOUNDATION
Allowlist, timeout, capture, store.

02 **Run comparison** — DIFF
Passing vs failing, evidence IDs.

03 **Evidence + hypothesis** — CLAIM
Claim hierarchy, confidence, unknown.

04 **Patch review + verification** — PROOF
Baseline check, apply gate, fresh run.

05 **Workspace + final report** — SHIP
One reviewable investigation record.

FINAL REPORT — 6 SECTIONS

- What ran
- What was observed
- What is hypothesized
- What remains unknown
- What changed
- What verification showed

THE REPORT IS THE PRODUCT · reviewable, linked, honest.

"The useful debugging tool isn't the one that sounds most certain. It's the one that makes the evidence impossible to ignore."

BREAKPOINT PRINCIPLE

Don't guess the bug.

Prove it.

END · 06 / 06